TECHNICAL DOCUMENTATION

Project Title:

RAG-Based Customer Support Assistant using LangGraph with HITL

INTRODUCTION

Retrieval-Augmented Generation (RAG) is an advanced approach that combines information retrieval with natural language generation to produce accurate, relevant, and context-aware responses. Traditional Large Language Models (LLMs) rely solely on pre-trained knowledge, which may be outdated or insufficient for domain-specific queries, often leading to hallucinations or incorrect answers. RAG overcomes this limitation by integrating an external knowledge source, where relevant information is retrieved in real time and provided as context to the model, thereby improving the accuracy and reliability of generated responses.

In this project, a RAG-based customer support assistant is designed to enhance the efficiency, scalability, and consistency of support systems. The system processes a PDF-based knowledge base, extracts and organizes the content through chunking, and converts it into embeddings for efficient semantic retrieval. When a user submits a query, the system retrieves the most relevant information from the stored data and uses an LLM to generate a context-aware response. This ensures that the answers are grounded in actual data rather than purely generated.

Furthermore, the system incorporates a graph-based workflow using LangGraph, which enables structured and stateful execution of the query-processing pipeline. This allows the system to dynamically control the flow of operations and make intelligent decisions based on intermediate results. To further improve reliability, a Human-in-the-Loop (HITL) mechanism is integrated, which handles scenarios where the system lacks confidence or encounters complex queries. By combining retrieval, generation, workflow orchestration, and human intervention, the proposed solution aims to deliver a robust and intelligent customer support system.

SYSTEM ARCHITECTURE EXPLANATION

The system is composed of multiple interconnected components that work together to process user queries and generate accurate, context-aware responses:

- **User Interface Layer:**
  Acts as the entry point of the system, allowing users to submit queries and view responses in a clear and user-friendly manner. It ensures smooth interaction between the user and the backend system.

- **Document Ingestion Module:**
  Responsible for loading PDF documents and extracting textual content. It converts unstructured document data into a structured format suitable for further processing and analysis.

- **Preprocessing Layer:**
  Performs basic cleaning operations such as removing unnecessary symbols, formatting text, and ensuring consistency in the extracted data before further processing.

- **Chunking Module:**
  Divides the extracted text into smaller, meaningful segments based on predefined chunk size and overlap. This helps maintain contextual continuity while improving retrieval efficiency.

- **Embedding System:**
  Converts each text chunk into high-dimensional vector representations using an embedding model. These embeddings capture semantic relationships between different pieces of text.

- **Vector Database (ChromaDB):**
  Stores embeddings along with their associated metadata and enables efficient indexing and fast similarity-based retrieval of relevant information.

- **Query Processing Module:**
  Accepts the user query, preprocesses it if necessary, and converts it into an embedding representation for comparison with stored data.

- **Retrieval Module:**
  Performs similarity search using the query embedding to retrieve the top-k most relevant chunks from the vector database, ensuring that only useful context is selected.

- **Context Aggregation Layer:**
  Combines retrieved chunks into a structured format that can be effectively used as input for the LLM, ensuring coherence and completeness of context.

- **LLM Processing Layer:**
  Takes the user query along with the retrieved context and generates a meaningful, accurate, and context-aware response using a Large Language Model.

- **Confidence Evaluation Module:**
  Assesses the quality and relevance of the generated response based on predefined criteria, such as retrieval quality or response completeness.

- **LangGraph Workflow Engine:**
  Orchestrates the entire process by managing nodes, edges, and state transitions. It enables structured execution and supports dynamic decision-making within the system.

- **Routing Layer:**
  Uses the output from the LLM and confidence evaluation to decide whether the response should be delivered directly or escalated for further handling.

- **Human-in-the-Loop (HITL) Module:**
  Handles cases where the system is unable to generate a reliable response. It forwards the query and context to a human agent and integrates the human-generated response back into the system.

- **Response Delivery Module:**
  Sends the final response, whether generated by the system or a human, back to the user through the interface in a clear and structured format.

- **Logging and Monitoring Module:**
  Tracks system performance, query handling, and errors. This helps in debugging, performance optimization, and improving system reliability over time.

- **End-to-End Pipeline Integration:**
  All components are interconnected and operate in a seamless pipeline, ensuring smooth data flow from document ingestion to final response delivery, while maintaining accuracy, efficiency, and scalability.

DESIGN DECISIONS

Chunk Size Selection

A moderate chunk size with overlap is chosen to balance context preservation and retrieval efficiency. Smaller chunks improve retrieval accuracy by narrowing down relevant information, while overlap ensures continuity of context across chunks and prevents loss of important information at boundaries. This balance helps the system maintain both precision and completeness during retrieval.

Embedding Strategy

Semantic embeddings are used to capture the meaning and context of text rather than relying on simple keyword matching. This allows the system to understand relationships between words and phrases, enabling more accurate and relevant retrieval even when the query wording differs from the original document text.

Vector Database Choice (ChromaDB)

ChromaDB is selected due to its lightweight nature, ease of integration, and efficient similarity search capabilities. It provides fast indexing and retrieval of embeddings, making it suitable for small to medium-scale applications. Additionally, it integrates well with modern AI frameworks, simplifying the development process.

Retrieval Approach

A top-k similarity search approach is used to retrieve the most relevant chunks based on the query embedding. This ensures that only the most useful and contextually relevant information is passed to the LLM. Limiting the number of retrieved chunks also helps reduce noise and improves the quality of the generated response.

Prompt Design Logic

The prompt is designed by combining the user query with the retrieved contextual information to guide the Large Language Model in generating accurate and relevant responses. The retrieved chunks are structured in a way that clearly provides supporting information, helping the model ground its answer in actual data rather than relying solely on pre-trained knowledge. Proper prompt formatting ensures that the LLM understands the task, maintains coherence, and avoids generating irrelevant or hallucinated content. Additionally, limiting the prompt to only the most relevant context helps improve response quality while reducing unnecessary computational overhead.

WORKFLOW EXPLANATION

The system follows a structured and stateful workflow managed by LangGraph, which enables controlled execution and dynamic decision-making throughout the query processing pipeline.

1. **User submits a query:**
   The process begins when the user provides a query through the interface, which serves as the input to the system.

2. **Query embedding generation:**
   The input query is converted into a vector representation using the embedding model, enabling semantic comparison with stored data.

3. **Retrieval of relevant chunks:**
   The query embedding is used to perform similarity search in ChromaDB, retrieving the top-k most relevant text chunks that match the query context.

4. **Context aggregation:**
   The retrieved chunks are combined and structured to form a meaningful context that can be effectively used by the LLM.

5. **Response generation using LLM:**
   The user query along with the retrieved context is passed to the Large Language Model, which generates a coherent and context-aware response.

6. **Confidence evaluation:**
   The generated response is evaluated based on predefined criteria such as relevance, completeness, and retrieval quality to determine its reliability.

7. **Conditional routing:**
   Based on the confidence score and evaluation, the system decides whether to return the response directly to the user or escalate the query.

8. **Final output or escalation:**
   - If the response is reliable, it is delivered to the user.
   - If confidence is low or context is insufficient, the query is forwarded to the Human-in-the-Loop (HITL) module for further handling.

LangGraph facilitates this entire workflow by defining nodes (processing steps), edges (flow between steps), and a shared state object that carries data across stages. This structured approach enables flexible, scalable, and dynamic execution of the system.

CONDITIONAL LOGIC

The system uses conditional logic to intelligently determine how each user query should be handled, ensuring both accuracy and reliability in the responses generated. After the response is produced by the LLM, it is evaluated based on factors such as relevance, completeness, and confidence score.

- **High confidence and relevant response:**
  If the generated answer is contextually accurate and meets the predefined confidence threshold, the system directly returns the response to the user, ensuring fast and efficient resolution.

- **Low confidence response:**
  If the confidence score is below the acceptable threshold, indicating uncertainty or potential inaccuracy, the system avoids providing a possibly incorrect answer and instead escalates the query to the Human-in-the-Loop (HITL) module.

- **No relevant context found:**
  If the retrieval module fails to identify meaningful or relevant chunks from the vector database, the system recognizes the lack of sufficient information and either returns a fallback response or escalates the query for human intervention.

- **Ambiguous or incomplete response:**
  In cases where the generated response is unclear, incomplete, or does not fully address the query, the system may trigger escalation to ensure the user receives a more accurate and helpful answer.

This conditional routing mechanism plays a critical role in maintaining system reliability, reducing incorrect outputs, and improving overall user trust by ensuring that uncertain cases are handled appropriately.

HUMAN-IN-THE-LOOP (HITL)

The Human-in-the-Loop (HITL) mechanism is integrated into the system to handle scenarios where the automated model is unable to generate a reliable or confident response. It acts as a critical fallback component that ensures the overall system maintains high accuracy and trustworthiness, especially when dealing with complex, ambiguous, or out-of-scope queries.

Role

- **Provides accurate responses for complex queries:**
  When the system encounters queries that require deeper understanding, domain expertise, or multi-step reasoning, the HITL module allows a human agent to analyze the situation and provide a precise response.
- **Acts as a fallback for system limitations:**
  It handles cases where the LLM fails due to low confidence, insufficient context, or retrieval issues, ensuring that the user still receives a meaningful answer.
- **Ensures quality control:**
  Human intervention helps validate responses and prevents the system from delivering incorrect or misleading information.

Benefits

- **Improves reliability and trust:**
  By involving human judgment in uncertain scenarios, the system minimizes incorrect outputs and builds user confidence.
- **Enhances response accuracy:**
  Complex queries that exceed the system's capability are handled effectively, leading to better overall performance.
- **Supports continuous improvement:**
  Human responses can be stored and analyzed to improve future system behavior, enabling better training and refinement of the model.

Limitations

- **Requires human availability:**
  The system depends on the presence of human agents, which may not always be feasible in real-time or large-scale deployments.
- I**ncreases response time:**
  Escalation to a human introduces delays compared to automated responses, affecting system speed.
- **Scalability challenges:**
  As the number of queries increases, relying on human intervention may become resource-intensive and harder to manage.

CHALLENGES & TRADE-OFFS

Retrieval Accuracy vs Speed

There is a trade-off between retrieval accuracy and system response time. Achieving higher accuracy often requires retrieving more chunks, using advanced similarity calculations, or applying complex ranking mechanisms, which can increase computational overhead and latency. On the other hand, optimizing for speed by limiting retrieval depth or simplifying search operations may reduce the quality and relevance of retrieved information. Therefore, a balance must be maintained to ensure both fast and accurate responses.

Chunk Size vs Context Quality

The choice of chunk size significantly impacts both retrieval performance and contextual understanding. Smaller chunks improve precision during retrieval, as they allow the system to pinpoint specific information more accurately. However, they may lack sufficient context, leading to incomplete or fragmented responses. Larger chunks preserve more context but may introduce irrelevant information, reducing retrieval precision. To address this, an optimal chunk size with overlap is used to maintain both context continuity and retrieval effectiveness.

Cost vs Performance

Using more advanced and powerful Large Language Models improves the quality, coherence, and accuracy of generated responses. However, this comes at a higher computational and financial cost, especially when using API-based models. On the other hand, using smaller or open-source models can reduce cost but may compromise performance. The system design must therefore balance cost constraints with the desired level of response quality and system efficiency.

TESTING STRATEGY

The system is evaluated using a variety of query types to ensure robust performance across different scenarios and use cases. Testing is carried out to validate both the accuracy of responses and the effectiveness of the overall workflow.

- **Simple queries (direct answers):**
  These include straightforward questions that can be directly answered using information present in the knowledge base. Testing with such queries ensures that the retrieval and response generation pipeline functions correctly for basic use cases.

- **Complex queries (multi-step reasoning):**
  These queries require combining information from multiple chunks or involve deeper understanding and reasoning. Testing with complex queries helps evaluate the system's ability to handle advanced scenarios and generate coherent, context-aware responses.

- **Out-of-scope queries:**
  These are queries that fall outside the provided knowledge base. Testing such cases ensures that the system correctly identifies the lack of relevant information and either provides a fallback response or triggers the Human-in-the-Loop (HITL) mechanism.

- **Edge case and ambiguous queries:**
  Additional testing is performed using unclear or partially defined queries to evaluate how well the system handles ambiguity and whether it appropriately escalates when necessary.

This comprehensive testing approach ensures that the system performs reliably, maintains accuracy, and handles different types of user inputs effectively under real-world conditions.

FUTURE ENHANCEMENTS

Support for multiple documents:

The system can be extended to handle multiple PDFs or diverse data sources simultaneously. This would allow the assistant to retrieve information from a broader knowledge base, improving its usefulness across different domains and use cases.

Integration of chat memory:

Adding conversational memory would enable the system to retain context across multiple user interactions. This would allow for more natural and continuous conversations, where the system can reference previous queries and responses.

Feedback-based learning:

The system can incorporate user feedback to improve its performance over time. By storing and analyzing feedback on responses, the model can be refined to provide more accurate and relevant answers in future interactions.

Deployment using web frameworks (e.g., Streamlit):

The system can be deployed as a web application using frameworks like Streamlit, enabling users to interact with it through a graphical interface. This enhances accessibility, usability, and real-world applicability of the solution.